

用户画像与长期记忆

从多源行为到可演化 FactMemory，再到静态、统计与动态兴趣画像

公司：高途教育集团 | 角色：负责人 | 时间：2026.02 - 2026.04 | 形态：V1 离线构建 + 本地展示

30 秒版本 我从 0 到 1 做了教育场景用户画像和长期记忆离线系统。先把搜索、课程、订单、社区和服务通话统一成 append-only HistoryEvent，再按 session/天/周聚合成 EventSummary，按周抽取并用 ADD/SKIP/MERGE/REPLACE 协议维护 FactMemory；画像分 L1 静态、L2 统计和 L3 最近兴趣。工程上做了规则前置、Prompt Hash 缓存、错误分类、Token 追踪和 EventSummary 并行，既控制成本又保留可追溯的记忆演化。

一、项目背景与目标

在线教育的用户信息分散在搜索、课程浏览、订单、社区互动、客服电话等系统。直接把原始行为塞给 LLM 会产生数据格式不一致、噪声大、成本高、事实重复/冲突、时效难治理等问题。本项目先建立统一中间层，再把稳定事实和动态兴趣分开建模，为搜索个性化和长期对话提供结构化输入。

层	回答的问题	更新特点
HistoryEvent	用户发生了什么最小行为？	append-only；保留事件时间与来源
EventSummary	一个 session/一天/一周发生了什么？	规则+LLM 窗口聚合；去噪压缩
FactMemory	哪些事实值得长期记住？	按 category+tag 增量演化，有 active/inactive
UserProfile	稳定属性、统计强度、近期兴趣是什么？	L1 规则、L2 统计、L3 LLM

二、我的工作

- 定义 HistoryEvent / EventSummary / FactMemory / UserProfile Schema 与端到端 Pipeline。
- 为搜索、课程、订单、社区、服务通话设计过滤、时间解析、窗口聚合和规则/LLM 分工。
- 设计 FactMemory 固定两级标签、TTL 与 ADD/SKIP/MERGE/REPLACE merge 协议，保留来源和 inactive 历史。
- 实现 L1 静态画像、L2 统计画像与 L3 最近四周兴趣画像，并设计搜索服务接入方式。

5. 建设缓存、重试、内容风控、Token/耗时 Trace、线程池并行和 Demo 数据集。

三、完整 Pipeline

多源 Parquet / 服务通话

-> [Rule] 清洗、过滤、时间统一 -> HistoryEvent

-> [Rule + LLM] session / 天 / 周聚合 -> EventSummary

-> [Rule] L1 Static + L2 Stat (含学科参与强度)

-> [LLM] 按周抽取候选 FactMemory

-> [Rule 前置 + LLM] ADD / SKIP / MERGE / REPLACE

-> [LLM] 最终 FactMemory + 最近 4 周摘要 -> L3 InterestProfile

-> DemoUserBundle JSON (构建与展示分离)

数据规模与处理边界

数据	规模	主要用途/过滤
AI 搜索	400,431 条	按 session 摘要；过滤无教育内容/风控失败
课程行为	3,222,414 条	近 91 天；有效 course_number 约 11.6%
订单	1,248,432 笔	按 ISO 周规则聚合；关联课程维表
社区互动	5,621,869 条	近 91 天；纯曝光跳过，有效互动约 1.7%
课程维表	1,035,669 门	补课程名/学科/年级，摘要覆盖率 98.7%
评论互动	177,844,530 条	当前不进入 Pipeline，避免误报处理规模

四、不同数据为何采用不同聚合

来源	窗口	方法	原因
AI 搜索	session	LLM 摘要	天然对话单元；保留用户问题与上下文
课程行为	天	LLM + 规则 fallback	浏览频繁，需要按日压缩并关联课程内容
订单	ISO 周	纯规则	金额/退款/课程是结构化事实，LLM 无增益
社区互动	天	LLM + 规则 fallback	需要结合动态文本判断教育主题
服务通话	一通一条	规则拼装 22 个结构字段	ASR 后端已结构化，避免重复 LLM

五、FactMemory 算法

5.1 为什么按 category + tag 分组

Merge 不应在全量记忆中任意比较，而是先按 category+tag 找 active 同组事实，缩小候选并保持业务含义一致。V1 固定 32 个 tag，基于 5 个种子用户 995 条 FactMemory 设计，覆盖率 90%+；长尾可进入兜底治理，但不能让 tag 无限膨胀。

5.2 Merge 协议

动作	适用情况	状态处理
SKIP	文本相同或已有事实完全覆盖	不新增；完全相同由规则直接判定
ADD	同组为空或新事实独立	新增 active；同组为空不调 LLM
MERGE	新旧互补	同组旧 active 全部 inactive；生成合并事实
REPLACE	新旧冲突/状态变化	同组旧 active 全部 inactive；新事实继承时效

- 6. 规则前置减少成本：完全相同直接 SKIP，同组为空直接 ADD，只有同组非空且文本不同才调 LLM。
- 7. MERGE/REPLACE 将全部同组旧记忆设为 inactive，并合并 source_summary_ids，保留演化链。
- 8. created_at 取事件窗口时间而非 Pipeline 运行时间，避免离线补跑污染时间语义。
- 9. TTL 由候选继承；trait 类型倾向 MERGE，状态变化才谨慎 REPLACE。

六、画像设计

画像层	实现	示例	为什么这样做
L1 StaticProfile	用户宽表规则映射	年龄/地区/年级/稳定偏好	确定性强，不需要 LLM
L2 StatProfile	订单/活跃/课程/标签统计	付费强度、AI 使用、学科参与度	可计算、可解释
L3 InterestProfile	最终 FactMemory + 最近 4 周摘要	近期兴趣、学习诉求、成长需求	动态语义复杂，适合 LLM

七、工程设计与收益

- 10. 构建/展示分离: build_all.py 调 LLM 产出 per-user JSON; Streamlit 只读 JSON, 零 LLM、秒级展示。
 - 11. Prompt 缓存键 = SHA-256(model + system + user prompt) 前 16 位, Prompt 或模型变化自动 miss。
 - 12. 最多重试 2 次: 网络、408/429/5xx 退避; 内容风控和其他客户端错误不重试。
 - 13. TokenTracker 区分等效总 token、实际 API token、缓存调用, 并按 event_summary/fact_extract/fact_merge/interest 分步统计。
 - 14. AI 搜索与社区 EventSummary 用 ThreadPoolExecutor(max_workers=4) 并行, 文档记录约 4 倍加速。
 - 15. 课程内容感知摘要覆盖率 98.7%; 32 个固定 FactMemory tag 覆盖率 90%+。
- V1 边界** 当前是离线 JSON, 无在线 API; 只聚焦 5 个种子用户构建, 50 个种子子集用于数据准备。服务化、Kafka、FactMemory embedding 与 EventSummary 在线 RAG 是规划, 不是已上线。

八、下游如何接 AI 搜索

产物	建议接入	原因
FactMemory	system context 全量注入	量少、长期稳定、信息密度高
L1/L2/L3 Profile	结构化 system context	直接描述用户稳定属性和近期兴趣
EventSummary	embedding 后按 Query RAG	数量随历史增长, 全量注入成本高且噪声大

九、面试高频问答

Q1. 为什么要三层记忆?

建议回答: HistoryEvent 保原子事实和溯源, EventSummary 做窗口压缩, FactMemory 承载可演化长期事实。直接从原始行为到长期记忆会让成本、噪声和更新逻辑耦合。

Q2. 为什么订单不用 LLM？

建议回答：金额、支付、退款、课程等字段结构化且规则可解释，LLM 增加成本和不确定性；只有需要理解文本语义的搜索/社区/课程内容才使用 LLM。

Q3. 为什么按周 merge？

建议回答：周是 V1 的成本和时效折中：降低每事件调用量，同时保留用户状态变化顺序。更实时可改天级或事件流，但需要增量服务和幂等。

Q4. 为什么 EventSummary 不做衰减？

建议回答：旧摘要在当期已经提炼进 FactMemory，后续只用于相关性 RAG；FactMemory 的 MERGE/REPLACE 承担事实演化，RAG 本身又按相关性筛选。

16. 追问展开：这依赖‘旧摘要不反复参与 fact 抽取’的增量合同。

Q5. 如何处理冲突事实？

建议回答：在同 category+tag 内让 LLM 判 REPLACE 或 MERGE；REPLACE 把旧 active 置 inactive 并保留来源，不能直接删除，否则无法审计演化。

Q6. 为什么规则前置？

建议回答：完全相同和无同组是确定性情形，直接 SKIP/ADD 可省大量 LLM 调用，也降低随机性。

Q7. TTL 怎么设计？

建议回答：明确时间锚点的备考阶段、年级、目标等可设 expires_at，再结合类别 base TTL；泛化表达宁可设，避免伪精确。更新时由新候选继承/覆盖。

Q8. 怎么避免 LLM 缓存污染？

建议回答：缓存 key 包含模型、system prompt 和 user prompt，内容任何变化都产生新 key；还要隔离环境/租户、记录版本，敏感内容不应长期明文缓存。

Q9. 线程池为什么能 4 倍？

建议回答：EventSummary 的多个 session/天任务主要等待外部 LLM I/O，线程池能并发隐藏等待；如果是 CPU 密集型 Python 计算，GIL 下线程未必线性加速。

Q10. 如何评估记忆质量？

建议回答：抽样评估事实正确性、完整性、冲突处理、时效、可溯源性和下游任务增益；离线可看 tag 覆盖 /merge action 分布，在线看检索命中、回答一致性和个性化收益。

Q11. 如何做隐私治理？

建议回答：源数据最小化、身份与内容分级、Prompt/日志脱敏、缓存 TTL、访问控制、删除传播；长期记忆要允许查看、纠正和删除，敏感属性限定用途。

Q12. V1 最大局限是什么？

建议回答：离线 per-user JSON 不能支撑在线新鲜度与高并发查询；下一步应服务化、做增量触发和存储索引，同时明确 EventSummary RAG 与 FactMemory 查询 SLA。

十、两分钟项目陈述模板

这个项目解决的是教育用户信息散落在搜索、课程、订单、社区和服务通话中，既无法直接用于个性化，也不适合把原始数据全塞给大模型。我从 0 到 1 设计了三层记忆和三层画像。底层先把不同表转成统一的 append-only HistoryEvent；再按数据语义用 session、天或周聚合成 EventSummary，其中订单等结构化数据走规则，搜索和社区文本走 LLM；最后按周抽取 FactMemory，并用 ADD、SKIP、MERGE、REPLACE 协议维护演化。为了控制成本，同组为空直接 ADD、文本相同直接 SKIP，只有真正冲突或补充才调用 LLM；旧事实不删除而是 inactive，并保留 source ids 和事件时间。画像则分 L1 静态规则、L2 统计和 L3 最近四周动态兴趣。工程上我做了 Prompt Hash 缓存、错误分类重试、Token 分步追踪和 EventSummary 线程池并行。资料记录课程内容关联覆盖率 98.7%、固定 32 标签覆盖率 90%+、并行约 4 倍加速。V1 是离线 JSON，不会把服务化和在线 RAG 说成已上线；下游设计是 FactMemory/Profile 直接注入，历史 EventSummary embedding 后按 Query 召回。

资料依据与表达边界

边界 可以明确说负责人、从 0 到 1 与离线 V1；不要说在线 API、Kafka、embedding/RAG 已上线。177M 评论数据未参与 Pipeline，不得并入处理量。

主要依据：

17. 高途实习材料/阚海_简历_27 应届.pdf（职责与项目名称的第一参考）
18. 高途实习材料/TECH_userprofile.md（实现、规模、收益和 V1 边界）

19. 高途实习材料/TECH_aisearch.md (下游上下文接入关系)